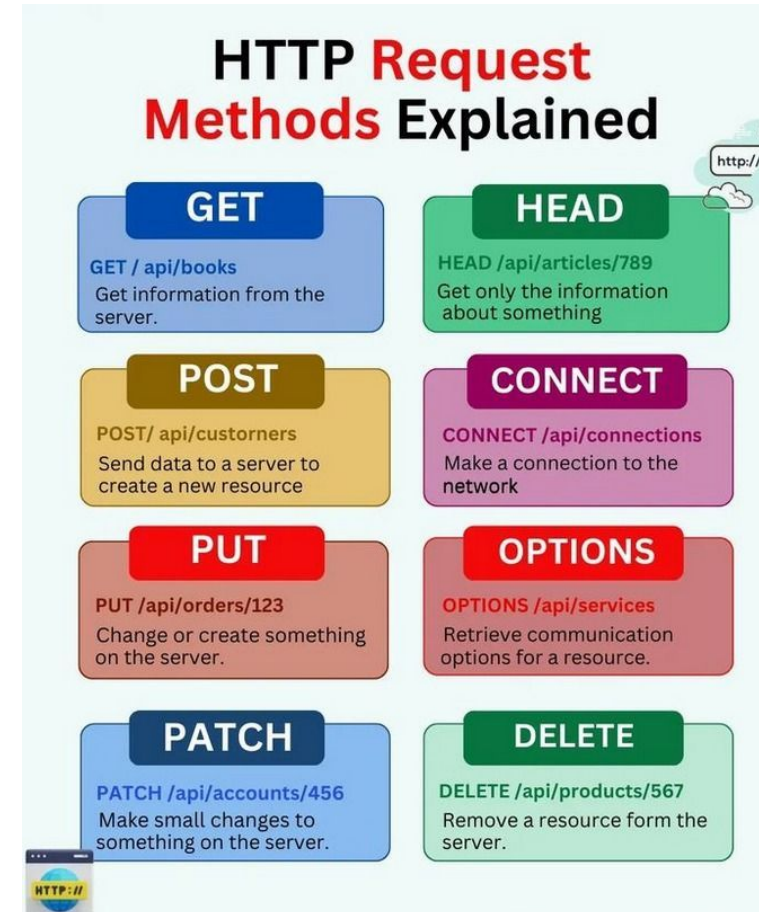


HTTP

HTTP Method

- HTTP defines a set of request methods to indicate the desired action to be performed for a given resource



HTTP request headers

- HTTP headers contain text information stored in key-value pairs, and they are included in every HTTP request
- These headers communicate core information, such as what browser the client is using and what data is being requested.

HTTP response headers

- An HTTP response comes with headers that convey important information such as the language and format of the data being sent in the response body.

▼ Response Headers

cache-control: private, max-age=0

content-encoding: br

content-type: text/html; charset=UTF-8

date: Thu, 21 Dec 2017 18:25:08 GMT

status: 200

strict-transport-security: max-age=86400

x-frame-options: SAMEORIGIN

HTTP status code

- HTTP status codes are 3-digit codes most often used to indicate whether an HTTP request has been successfully completed.
- Status codes are broken into the following 5 blocks:
 1. 1xx Informational
 2. 2xx Success
 3. 3xx Redirection
 4. 4xx Client Error
 5. 5xx Server Error
- Status codes starting with the number '2' indicate a success.
 - For example, after a client requests a webpage, the most commonly seen responses have a status code of '200 OK', indicating that the request was properly completed.
- If the response starts with a '4' or a '5' that means there was an error and the webpage will not be displayed.
- A status code that begins with a '4' indicates a client-side error
- A status code beginning in '5' means something went wrong on the server side.

Cookies

- An Http cookie, also known as a web cookie, is a small piece of data that the server sends to the web browser.
- The typical use of the cookies is session management, personalization and tracking.
- In all these cases it is used to identify multiple requests from the same resource (browser).
- It helps the user to keep logged in and trace his data, such as user name and session.

How Cookies Work

1. When a user visits a website, the web server — which stores the website's data — sends a short stream of identifying info which is the cookie- to the visiting user's web browser.
2. Cookies are stored locally in the web browser to remember the “name-value pair” that identifies the user. The server sends the cookie when it wants the web browser to save it.
3. If a user returns to that site in the future, the web browser returns that data to the web server in the form of a cookie to recall the data from previous sessions and to identify the user

HTTP: A stateless protocol

- A stateless protocol is a communication protocol in which the receiver must not retain session state from previous requests.
- HTTP is a "stateless" protocol means that each time a client retrieves a Webpage, the client or the web browser opens a separate connection to the Web server
- The server automatically does not keep any record of previous client request if it is using HTTP

Cookie Attributes

Secure Attribute: The Secure attribute tells the browser to only send the cookie if the request is being sent over a secure channel such as HTTPS. This will help protect the cookie from being passed in unencrypted requests.

HttpOnly Attribute: The HttpOnly attribute is used to help prevent attacks such as session leakage, since it does not allow the cookie to be accessed via a client-side script such as JavaScript.

Domain Attribute: The Domain attribute is used to compare the cookie's domain against the domain of the server for which the HTTP request is being made. If the domain matches or if it is a subdomain, then the path attribute will be checked next.

Path Attribute: In addition to the domain, the URL path that the cookie is valid for can be specified using path attribute and if the domain and path match, then the cookie will be sent in the request.

Expires Attribute: The Expires attribute is used to set persistent cookies and limit lifespan if a session lives for too long

SameSite Attribute: This attribute is used to assert that a cookie should not to be sent along with cross-site requests and helps mitigate the risk of cross-origin information leakage and also to prevent cross-site request forgery attacks. This attribute can be configured in three different modes: Strict, Lax and None



Cookie Prefixes

In order to give cookie the capabilities to guarantee the integrity and confidentiality of the information stored in them concept of Cookie Name Prefixes was introduced.

Host Prefix: The __Host- prefix expects cookies to fulfill the following conditions:

- The cookie must be set with the Secure attribute.
- The cookie must be set from a URI considered secure by the user agent.
- Sent only to the host who set the cookie and MUST NOT include any Domain attribute.
- The cookie must be set with the Path attribute with a value of / so it would be sent with every request to the host.

Secure Prefix: The __Secure- prefix is less restrictive. Any cookie that matches the prefix __Secure- would be expected to fulfill the following conditions:

- The cookie must be set with the Secure attribute.
- The cookie must be set from a URI considered secure by the user agent.



Examples

An example Cookie Attribute Configuration can be:

Set-Cookie: __Host-SID=<session token>; path=/; Secure; HttpOnly; SameSite=Strict.

Set-Cookie: __Host-SID=12345; Secure; Path=

Invalid Settings for Cookies would include:

Set-Cookie: __Host-SID=12345

Set-Cookie: __Host-SID=12345;Secure

Set-Cookie: __Host-SID=12345; Domain=site.example

Set-Cookie: __Host-SID=12345; Domain=site.example; Path=

Set-Cookie: __Host-SID=12345; Secure; Domain=site.example; Path=



Cookies

Session Cookies

These cookies are temporary and will only be stored in the memory of user's browser while it's open. When it's closed, the cookie will be removed from browser's history, and, because of this, they are a lower security risk.

These cookies are often used for personalization when working within e-commerce shopping carts, controlling what a user sees on a page when they're conducting a one-off, multi-page site visit, or for other storage means that are short-term.

Persistent Cookies

These cookies are used over a much longer period of time, as an expiration date is tagged to them by the issuer using the expires attribute. Each time user return to the website that created this cookie, or go to a website that has a resource produced by the cookie's issuer (eg, an ad), this data is returned to the user.

Thus, these cookies can track user's browsing activity not just on the original site where the cookie was created but on other sites that have a resource which has been produced by the original site. For example, Facebook and Google use these kinds of mechanisms to create a user activity log across a range of different websites. So, when the user click "Remember Me" (or something similar) after they have logged into online account somewhere, they will be creating a persistent cookie, which will store the login details on the browser.

As these persistent cookies are present for a longer period than session cookies, and can essentially track what the user is doing over more than one site, a greater security risk is posed by them.



Same Origin Policy: How a Web Page works

- A web page typically has a domain name (for example www.amrita.edu)
- The browser then sends a request to that server for the page you want, using a standard called Hypertext Transfer Protocol or HTTP or HTTPs
- HTTP communicates through TCP ports and the default port used by HTTP is 80 and by HTTPs is 443.



Same Origin Policy

- Same Origin Policy is a critical web application security mechanism
- The same-origin policy states that when a user is viewing a web page in his browser, script running on that web page should only be able to read from or write to the content of another web page if both pages have the same origin.



Same Origin Policy

- The origin consists of combination of :
 - page's application layer protocol (HTTP or HTTPS),
 - its TCP port (usually 80 for HTTP or 443 for HTTPS),
 - its domain name



Same Origin Policy

Source URL

URL

<http://www.amrita.edu/photos>

<https://amritaedu/photos>

<https://www.amrita.edu:8080/photos>

<https://www.amrita.edu/gallery>

<https://www.amrita.edu/photos>

Scripting Requests Allowed?

Will not work; protocol is different

Will not work; domain name is different

Will not work; port number is different

Will work



Same Origin Policy

- It limits functionality of scripting code running in users' web browser or at the client side
- It applies to browsers running client side scripting code and has no effect on what server side code can access



Need for SOP

- Without the same-origin policy, if you visit a malicious website the script in that website would be able to read confidential information from other websites where you have already logged in.
 - For example: abc.com would be able to read information from gmail.com if there was no SOP.
- When a browser sends an HTTP request from one origin to another, any cookies, including authentication session cookies, relevant to the other domain are also sent as part of the request.
- Essentially SOP is required for privacy and security

